



Marwadi
University
Marwadi Chandarana Group

NAAC



MARWADI UNIVERSITY

FACULTY OF COMPUTER APPLICATIONS

Programming with Python & Problem Solving (05MC0107)

LAB MANUAL

Programming with Python & Problem Solving (05MC0107) Lab Manual (2026-27)

Program: MCA

Semester: 1

Name of the Student: SAMIR SAJIDBHAI SUMRA

Enrolment No.: 92600584103

INDEX

Lab	Program	Date	Remarks	Signature
1	Write a program to demonstrate basic input and output operations in Python.	17/08/2026		
2	Write a program to illustrate the use of different data types and type casting.	17/08/2026		
3	Write a program to perform arithmetic relational and logical operations using Python operators.	17/08/2026		
4	Write a program to demonstrate string operations including slicing formatting and built-in string functions.	17/08/2026		
5	Write a program to create and manipulate lists using indexing slicing and list comprehensions.	17/08/2026		
6	Write a program to illustrate the use of tuples and sets with basic operations.	17/08/2026		
7	Write a program to create a dictionary and demonstrate dictionary methods and iteration.	17/08/2026		
8	Write a program to explain mutable and immutable objects in Python.	17/08/2026		
9	Write a program to define and use user-defined functions with different types of arguments.	17/08/2026		
10	Write a program to demonstrate recursion using factorial or Fibonacci series.	17/08/2026		
11	Write a program to demonstrate conditional statements using if-else and if-elif-else.	30/08/2026		
12	Write a program to check whether a number is positive negative or zero using nested conditions.	30/08/2026		
13	Write a program to generate a multiplication table using a for loop.	30/08/2026		
14	Write a program to find the sum of digits of a number using a while loop.	30/08/2026		
15	Write a program to demonstrate the use of break continue and pass statements.	30/08/2026		
16	Write a program to iterate over lists strings and dictionaries using loops.	30/08/2026		
17	Write a program to demonstrate list dictionary and set comprehensions.	30/08/2026		
18	Write a program to illustrate variable scope using local global and nonlocal variables.	30/08/2026		
19	Write a program to demonstrate iterators and iterables in Python.	30/08/2026		
20	Write a program to generate a sequence of numbers using generator functions and yield keyword.	30/08/2026		

21				
22				
23				
24				
25				
26				
27				
28				
29				
30				
31				
32				
33				
34				
35				
36				

Practical No. 11

Definition:

Write a program to demonstrate conditional statements using if-else and if-elif-else.

Aim / Objectives:

- To understand the concept of conditional statements in Python.
- To demonstrate the use of the if statement.
- To demonstrate the use of the if-else statement.
- To demonstrate the use of the if-elif-else statement.
- To learn how to execute different statements based on given conditions.

Source Code:

```
# Using if statement
age = 20
if age >= 18:
    print("You are eligible to vote.")

# Using if-else statement
number = 10
if number % 2 == 0:
    print("Number is Even")
else:
    print("Number is Odd")

# Using if-elif-else statement
marks = 75

if marks >= 90:
    print("Grade: A")
elif marks >= 70:
    print("Grade: B")
elif marks >= 50:
    print("Grade: C")
```

else:

```
print("Grade: F")
```

Output:

```
You are eligible to vote.  
Number is Even  
Grade: B
```

Conclusion:

Thus, the Python program successfully demonstrates the use of if, if-else, and if-elif-else conditional statements.

Practical No.12

Definition: Write a program to check whether a number is positive negative or zero using nested conditions.

Aim / Objectives:

- To understand the concept of nested conditional statements in Python.
- To write a program to check whether a number is positive, negative, or zero.
- To use if and else statements inside another conditional statement.
- To learn how to apply multiple conditions to a single problem.
- To display the appropriate result based on the entered number.

Source Code:

```
num = int(input("Enter a number: "))
```

```
if num >= 0:
```

```
    if num == 0:
```

```
        print("Number is Zero")
```

```
    else:
```

```
        print("Number is Positive")
```

```
else:
```

```
    print("Number is Negative")
```

Output:

```
Enter a number: 22  
Number is Positive
```

Conclusion:

Thus, the program successfully checks whether the given number is **positive, negative, or zero** using **nested conditional statements** in Python.

Practical No.13

Definition: Write a program to generate a multiplication table using a for loop.

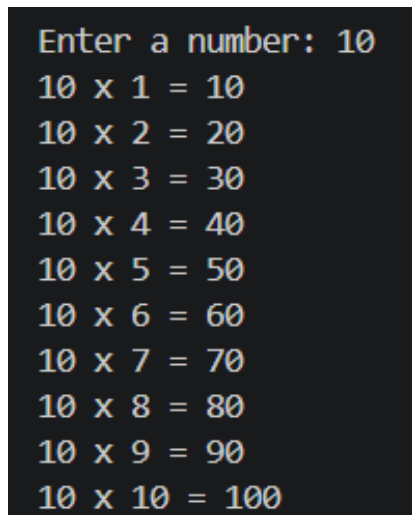
Aim / Objectives:

- To understand the concept of for loop in Python.
- To write a program to generate a multiplication table.
- To learn how to use the range() function with a for loop.
- To perform repeated multiplication using iteration.
- To display the multiplication table in a proper format.

Source Code:

```
num = int(input("Enter a number: "))  
  
for i in range(1, 11):  
    print(num, "x", i, "=", num * i)
```

Output:



```
Enter a number: 10  
10 x 1 = 10  
10 x 2 = 20  
10 x 3 = 30  
10 x 4 = 40  
10 x 5 = 50  
10 x 6 = 60  
10 x 7 = 70  
10 x 8 = 80  
10 x 9 = 90  
10 x 10 = 100
```

Conclusion:

Thus, the program successfully generates the multiplication table of a given number using a for loop in Python.

Practical No.14

Definition: Write a program to find the sum of digits of a number using a while loop.

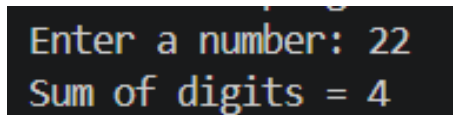
Aim / Objectives:

- To understand the concept of the while loop in Python.
- To write a program to find the sum of digits of a number.
- To learn how to extract individual digits from a number.
- To perform repeated operations using a while loop.
- To display the sum of all digits of the given number.

Source Code:

```
num = int(input("Enter a number: "))  
sum = 0  
while num > 0:  
    digit = num % 10  
    sum = sum + digit  
    num = num // 10  
print("Sum of digits =", sum)
```

Output:



```
Enter a number: 22  
Sum of digits = 4
```

Conclusion:

Thus, the program successfully finds the **sum of digits of a given number using a while loop** in Python.

Practical No.15

Definition: Write a program to demonstrate the use of break continue and pass statements.

Aim / Objectives:

- To understand the concept of break, continue, and pass statements in Python.
- To demonstrate the use of the break statement.
- To demonstrate the use of the continue statement.
- To demonstrate the use of the pass statement.
- To understand how these statements control the flow of a program.

Source Code:

1. Break statement

```
print("Using break:")
```

```
for i in range(1, 6):
```

```
    if i == 4:
```

```
        break
```

```
    print(i)
```

2. Continue statement

```
print("\nUsing continue:")
```

```
for i in range(1, 6):
```

```
    if i == 3:
```

```
        continue
```

```
    print(i)
```

3. Pass statement

```
print("\nUsing pass:")
```

```
for i in range(1, 6):
```

```
    if i == 3:
```

```
        pass
```

```
    print(i)
```

Output:

```
Using break:
1
2
3

Using continue:
1
2
4
5

Using pass:
1
2
3
4
5
```

Conclusion:

Thus, the program successfully demonstrates the use of **break, continue, and pass statements** in Python to control the flow of execution.

Practical No.16

Definition: Write a program to iterate over lists strings and dictionaries using loops.

Aim / Objectives:

- To understand how loops are used to iterate over different data types in Python.
- To iterate over elements of a **list** using a for loop.
- To iterate over characters of a **string** using a loop.
- To iterate over keys and values of a **dictionary** using a loop.
- To understand the practical use of loops with Python collections.

Source Code:

1. Iterate over a List

```
print("List Elements:")
```

```
fruits = ["Apple", "Banana", "Mango", "Orange"]
```

```
for fruit in fruits:
```

```
    print(fruit)
```

2. Iterate over a String

```
print("\nString Characters:")
```

```
name = "Python"
```

```
for ch in name:
```

```
    print(ch)
```

3. Iterate over a Dictionary

```
print("\nDictionary Elements:")
```

```
student = {
```

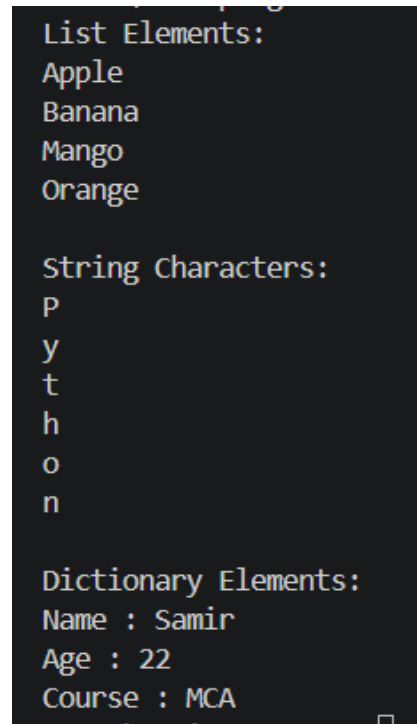
```
    "Name": "Samir",
```

```
    "Age": 22,
```

```
    "Course": "MCA"
```

```
}  
  
for key, value in student.items():  
    print(key, ":", value)
```

Output:



```
List Elements:  
Apple  
Banana  
Mango  
Orange  
  
String Characters:  
P  
y  
t  
h  
o  
n  
  
Dictionary Elements:  
Name : Samir  
Age : 22  
Course : MCA
```

Conclusion:

Thus, the program successfully demonstrates how to **iterate over lists, strings, and dictionaries using loops in Python.**

Practical No.17

Definition: Write a program to demonstrate list dictionary and set comprehensions.

Aim / Objectives:

- To understand the concept of comprehensions in Python.
- To demonstrate list comprehension for creating lists.
- To demonstrate dictionary comprehension for creating dictionaries.
- To demonstrate set comprehension for creating sets.
- To learn how comprehensions make Python programs shorter and simpler.

Source Code:

```
# 1. List Comprehension  
  
numbers = [1, 2, 3, 4, 5]
```

```
squares = [x * x for x in numbers]
print("List Comprehension:")
print(squares)
```

2. Dictionary Comprehension

```
numbers = [1, 2, 3, 4, 5]
square_dict = {x: x * x for x in numbers}
```

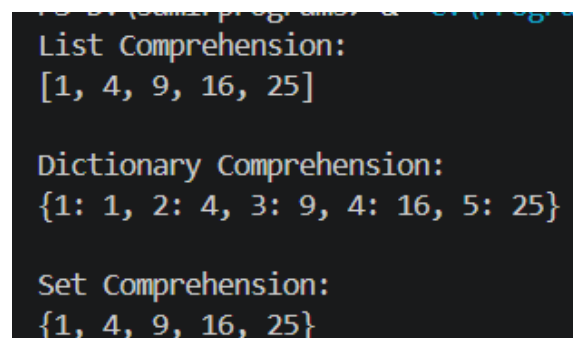
```
print("\nDictionary Comprehension:")
print(square_dict)
```

3. Set Comprehension

```
numbers = [1, 2, 2, 3, 3, 4, 5]
square_set = {x * x for x in numbers}
```

```
print("\nSet Comprehension:")
print(square_set)
```

Output:



```
List Comprehension:
[1, 4, 9, 16, 25]

Dictionary Comprehension:
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Set Comprehension:
{1, 4, 9, 16, 25}
```

Conclusion:

Thus, the program successfully demonstrates **list, dictionary, and set comprehensions** in Python and shows how they can be used to create collections in a concise manner.

Practical No.18

Definition: Write a program to illustrate variable scope using local global and nonlocal variables.

Aim / Objectives:

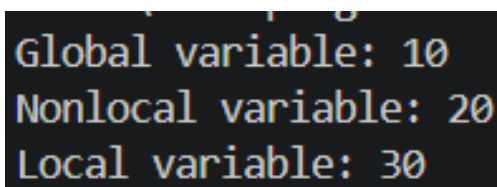
- To understand the concept of variable scope in Python.
- To demonstrate the use of **local variables**.
- To demonstrate the use of **global variables**.
- To demonstrate the use of **nonlocal variables**.
- To understand how variables are accessed and modified in different scopes.

Source Code:

```
# Global variable
x = 10

def outer():
    # Nonlocal variable
    y = 20
    def inner():
        # Local variable
        z = 30
        print("Global variable:", x)
        print("Nonlocal variable:", y)
        print("Local variable:", z)
    inner()
outer()
```

Output:



```
Global variable: 10
Nonlocal variable: 20
Local variable: 30
```

Conclusion:

Thus, the program successfully demonstrates **local, global, and nonlocal variable scopes** in Python.

Practical No.19

Definition: Write a program to demonstrate iterators and iterables in Python.

Aim / Objectives:

- To understand the concept of **iterables and iterators** in Python.
- To demonstrate how an iterable can be converted into an iterator.
- To use the `iter()` function to create an iterator.
- To use the `next()` function to access elements one by one.
- To understand the difference between an iterable and an iterator.

Source Code:

```
# Iterable

numbers = [10, 20, 30, 40, 50]

print("Iterable:")

for num in numbers:

    print(num)

# Convert iterable into iterator

iterator = iter(numbers)

print("\nIterator:")

print(next(iterator))

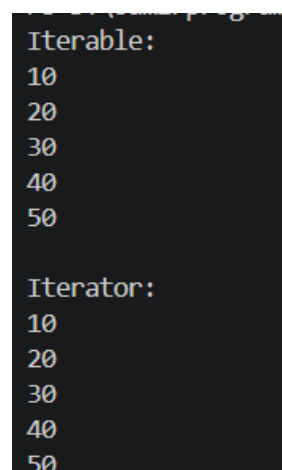
print(next(iterator))

print(next(iterator))

print(next(iterator))

print(next(iterator))
```

Output:



```
Iterable:
10
20
30
40
50

Iterator:
10
20
30
40
50
```

Conclusion:

Thus, the program successfully demonstrates **iterables and iterators** in Python using the `iter()` and `next()` functions.

Practical No.20

Definition: Write a program to generate a sequence of numbers using generator functions and `yield` keyword.

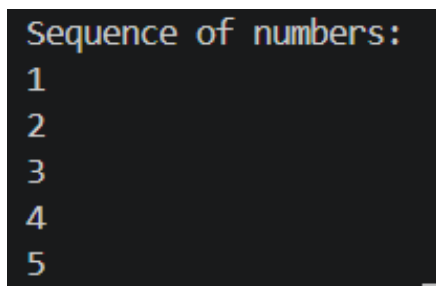
Aim / Objectives:

- To understand the concept of **generator functions** in Python.
- To learn the use of the **yield keyword**.
- To generate a sequence of numbers using a generator.
- To understand how generators produce values one at a time.
- To demonstrate memory-efficient sequence generation in Python.

Source Code:=

```
def generate_numbers(n):  
    for i in range(1, n + 1):  
        yield i  
  
# Create generator  
numbers = generate_numbers(5)  
print("Sequence of numbers:")  
for num in numbers:  
    print(num)
```

Output:



```
Sequence of numbers:  
1  
2  
3  
4  
5
```

Conclusion:

Thus, the program successfully generates a sequence of numbers using a **generator function and the yield keyword** in Python.